

파이썬데이터분석



데이터사이언스 전공

담당교수: 서지훈



강남대학교

1. 파이썬 기반 복습

1.1 기계 학습의 이해

- 경험을 통해서 나중에 유사하거나 같은 일(task)를 더 효율적으로 처리할 수 있도록 시스템의 구조나 파라미터를 바꾸는 것
- 컴퓨터가 지식을 갖게 만드는 작업

(1) 지도학습

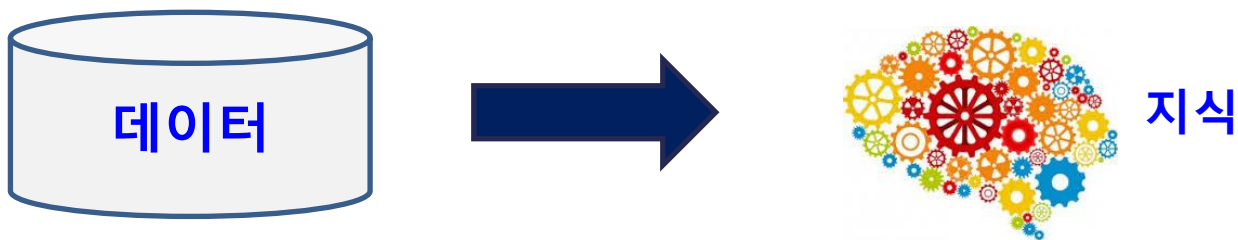
- 입력과 대응하는 출력을 데이터로 제공하고 대응관계의 함수 찾기

(2) 비지도학습

- 데이터만 주어진 상태에서 유사한 것들을 서로 묶어 군집을 찾거나 확률분포 표현

(3) 강화학습

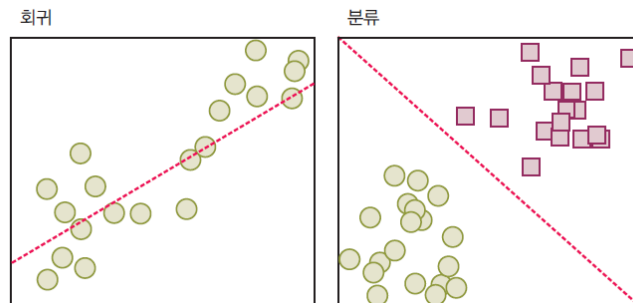
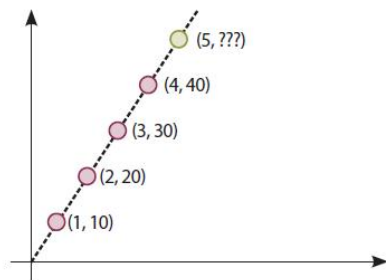
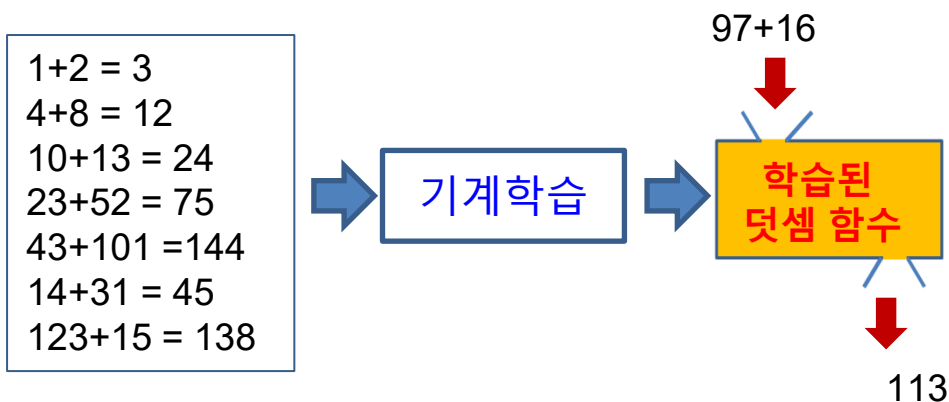
- 상황 별 행동에 따른 시스템의 보상 값(reward value)만을 이용하여, 시스템에 대한 바람직한 행동 정책(policy) 찾기



1. 파이썬 기반 복습

1.2 기계 학습의 종류 상세

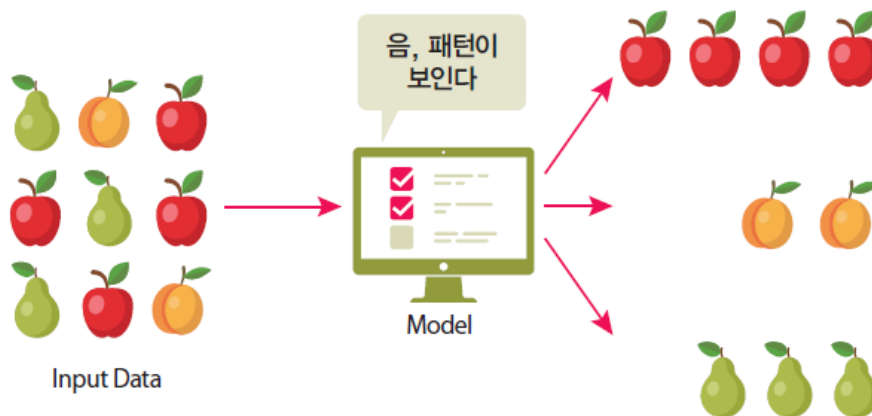
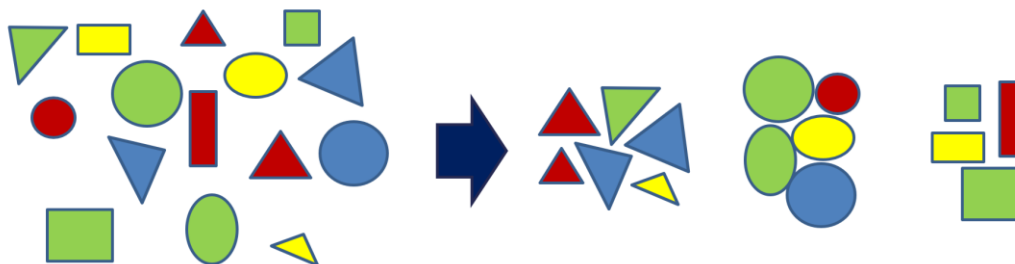
- 지도학습(supervised learning)은 입력(문제)과 대응하는 출력(답)을 데이터로 제공하고 대응관계의 함수 또는 패턴을 찾는 것
- 즉, 주어진 입력-출력 쌍을 학습한 후에 새로운 입력 값이 들어왔을 때, 합리적인 출력 값을 예측함
- 예시) 분류, 회귀



1. 파이썬 기반 복습

1.2 기계 학습의 종류 상세

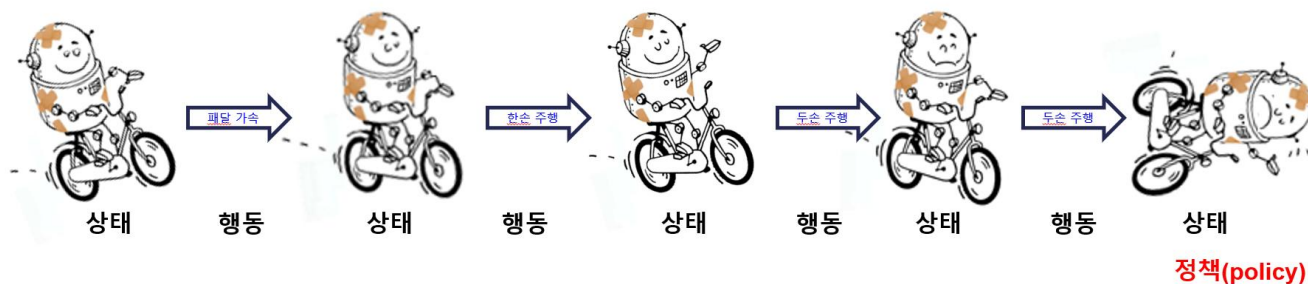
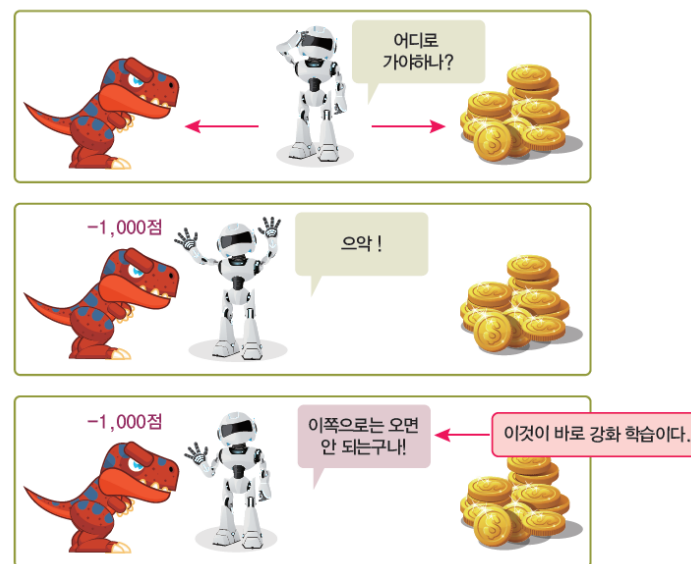
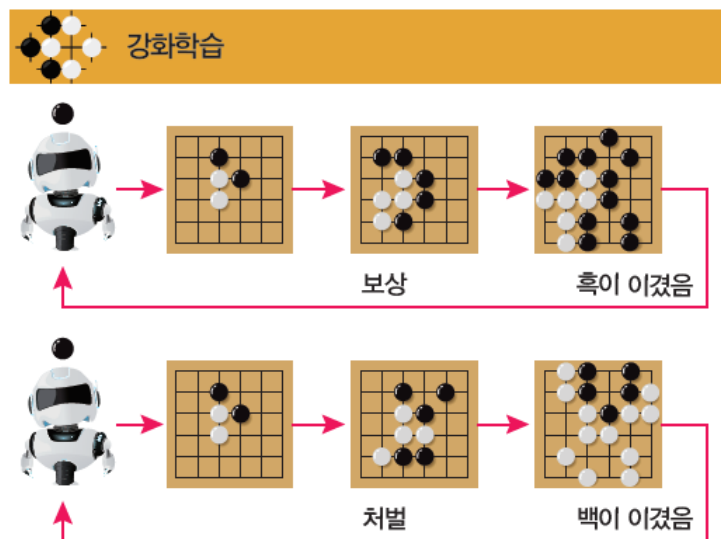
- 비지도학습(unsupervised learning)은 답이 없는 문제들만 있는 데이터들로 부터 패턴을 추출하는 것
- 결과적으로 “식 $y = f(x)$ 에서 레이블 y 가 주어지지 않는 것”
- 즉, “교사” 없이 컴퓨터가 스스로 입력들을 분류하는 것을 의미함.
- 예시) 군집화, 밀도 추정, 토픽 모델링



1. 파이썬 기반 복습

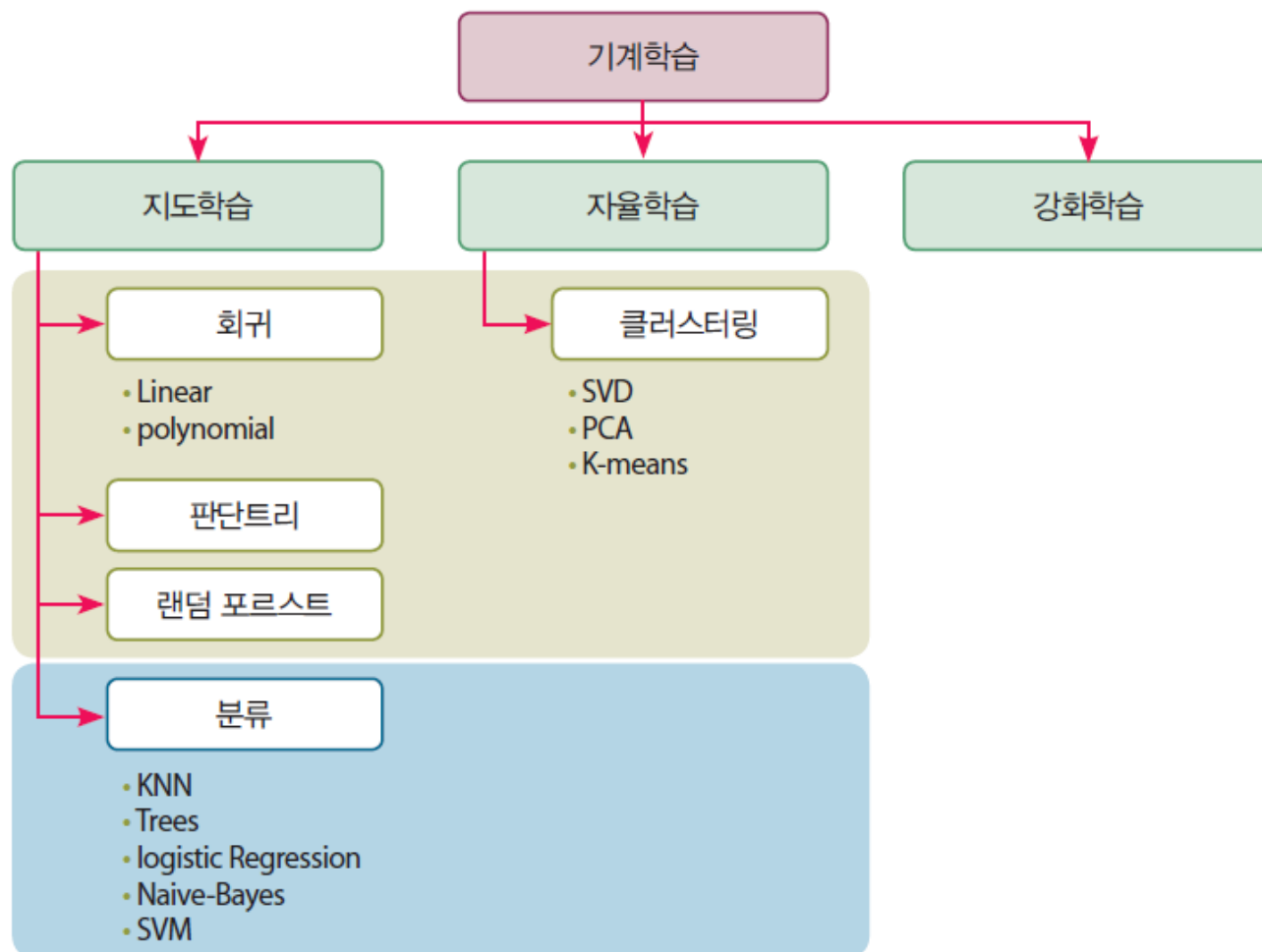
1.2 기계 학습의 종류 상세

- 강화학습(reinforcement learning)은 문제에 대한 직접적인 답을 주지는 않지만 경험을 통해 기대 보상(expected reward)이 최대가 되는 정책을 찾는 학습 방법



1. 파이썬 기반 복습

1.3 기계 학습의 전체 구성

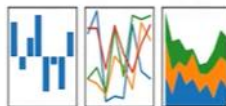
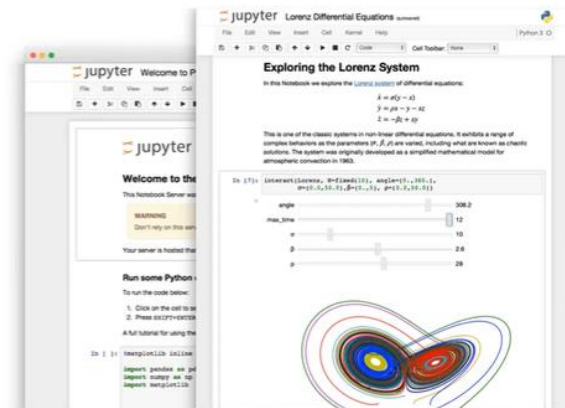


1. 파이썬 기반 복습

1.4 파이썬 환경

주피터 노트북은 대표적인 대화형 파이썬 툴입니다. 대화형 툴이라는 말은 마치 학교에서 선생님이 학생들에게 설명하듯이 프로그래밍과 이에 대한 설명적인 요소를 결합했다는 뜻입니다. 전체 프로그램에서 특정 코드 영역별로 개별 수행을 지원하므로 영역별로 코드 이해가 매우 명확하게 설명될 수 있습니다.

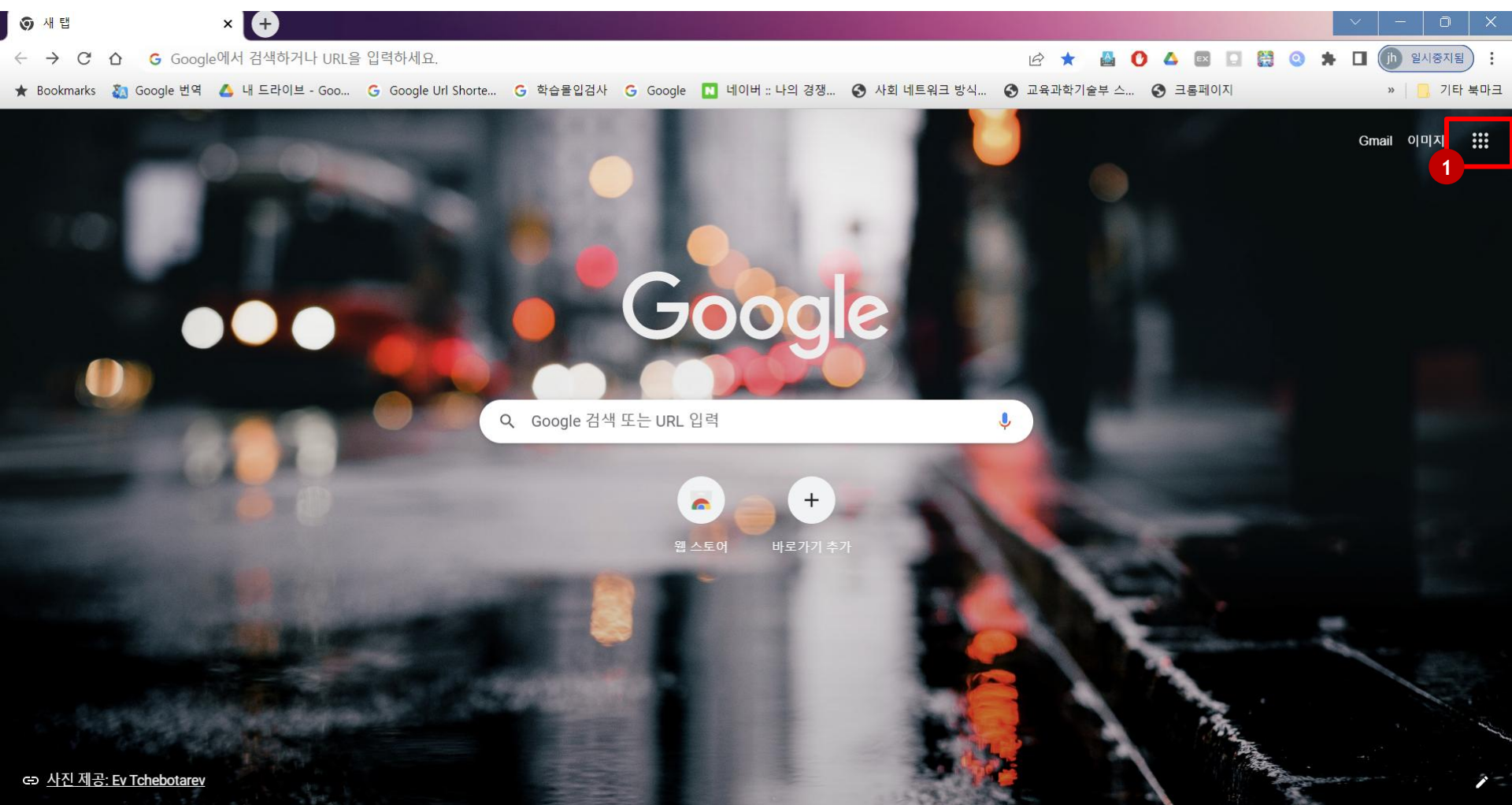
노트북(Notebook), 즉 '공책'이라는 접미사에서 유추할 수 있듯이 학생들이 필기하듯이 중요 코드 단위로 설명을 적고 코드를 수행해 그 결과를 볼 수 있게 만들어서 직관적으로 어떤 코드가 어떤 역할을 하는지 매우 쉽게 이해할 수 있도록 지원합니다.



- 머신러닝 애플리케이션 구현에서 다양한 데이터의 추출/가공/변환이 상당한 영역을 차지하고 데이터 처리 부분은 대부분 넘파이와 판다스의 몫.
- 사이킷런이 넘파이 기반에서 작성됐기 때문에 넘파이의 기본 프레임워크를 이해하지 못하면 사이킷런 역시 실제 구현에서 많은 벽에 부딪힐 수 있음
- 사이킷런은 API 구성이 매우 간결하고 직관적이어서 이를 이용한 개발 또한 상대적으로 쉽지만 넘파이와 판다스 API는 더 방대하기 때문에 이를 익히는 데 시간이 많이 소모 될 수 있음. 하지만 머신러닝을 위해서 이들을 많은 시간을 들여 전문적으로 공부하는 것은 효율적이지 못함.
- 넘파이와 판다스에 대한 기본 프레임워크와 중요 API만 습득하고, 일단 코드와 부딪쳐 가면서 모르는 API에 대해서는 인터넷 자료를 통해 채득하는 것이 머신러닝뿐만 아니라 넘파이와 판다스에 관한 이해를 넓히는 더 빠른 방법임.

1. 파이썬 기반 복습

1.4 파이썬 환경



1. 가급적 크롬 브라우저를 이용하세요.
2. 메인 화면에서 오른쪽 상단 그리드(바둑판 모양)을 클릭하세요.

1. 파이썬 기반 복습

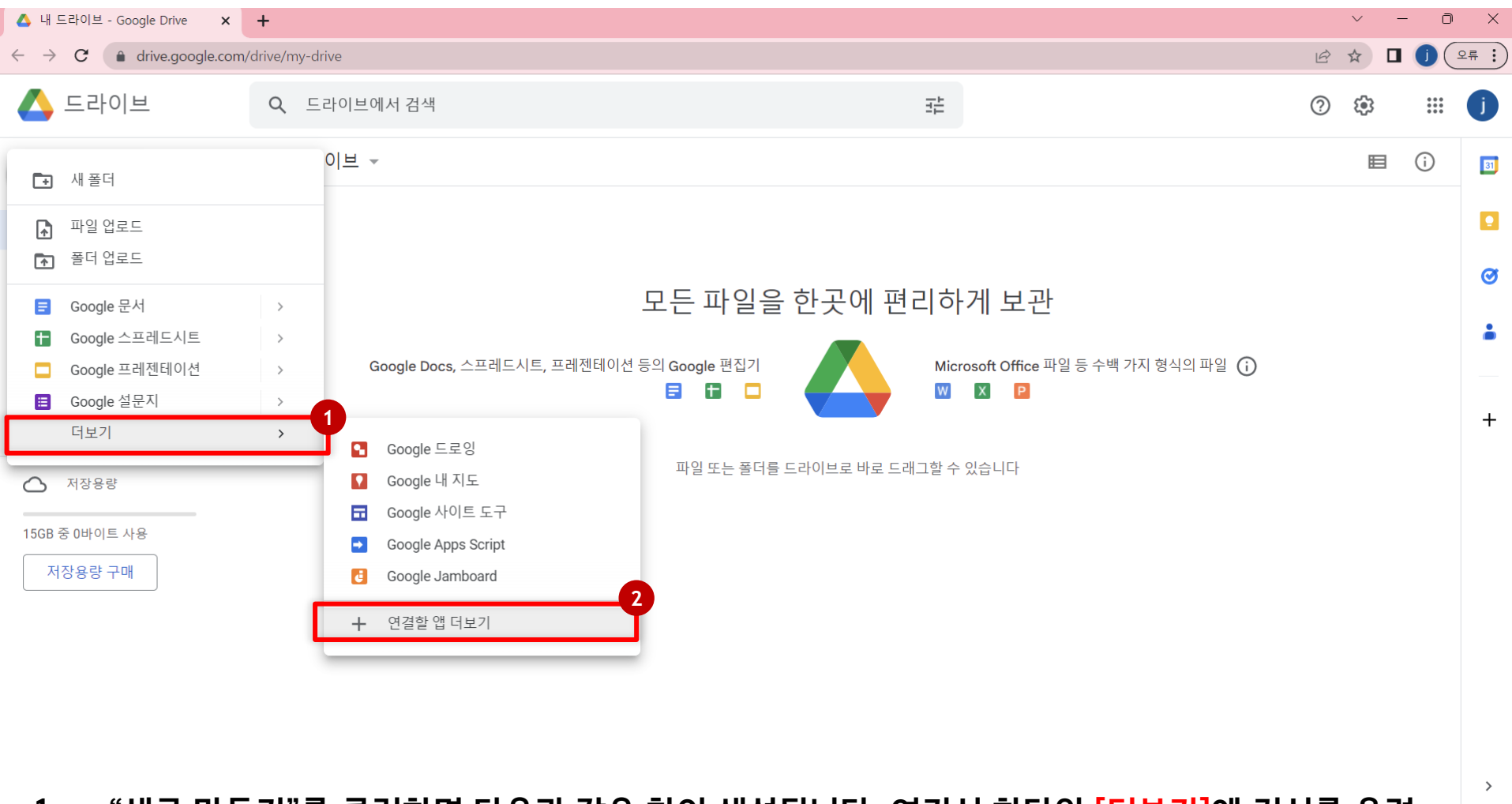
1.4 파이썬 환경

The screenshot shows the Google Drive web interface. On the left sidebar, the '새로 만들기' (New) button is highlighted with a red box and a red circle containing the number '1'. On the right side, the '내 드라이브' (My Drive) panel is open, and its close button (an 'X' icon) is circled in red. A text annotation 'X 버튼으로 닫아도 괜찮아요.' (It's okay to close with the X button.) is placed next to the close button. The main area of the drive shows a message about organizing files and lists supported file formats like Google Docs and Microsoft Office files.

1. 드라이브 메인 화면에서 왼쪽 상단 “새로 만들기 ” 를 클릭하세요.
2. 오른쪽 “내 드라이브” 창은 닫아도 괜찮아요.

1. 파이썬 기반 복습

1.4 파이썬 환경



1. “새로 만들기”를 클릭하면 다음과 같은 창이 생성됩니다. 여기서 하단의 **[더보기]**에 커서를 올려 놓습니다.
2. 다음으로 **+ 연결할 앱 더보기**를 클릭합니다.

1. 파이썬 기반 복습

1.4 파이썬 환경

Google Workspace Marketplace

1 Colaboratory

모든 필터 호환 기기: 가격

검색결과: Colaboratory

Google에서는 리뷰나 평점을 확인하지 않습니다. [리뷰 및 결과에 대해 자세히 알아보기](#)

2

Colaboratory
Colaboratory team
This allows Google Colaboratory to open and create files in Google Drive. It i...

★ 4.7 ↓ 10,000,000+

MAIL MERGE for SHEETS
yamm
YAMM - Best mailing tool
Talarian
The best and easiest mail merge tool for Gmail. Send mass emails with high deliverability directly...

★ 4.7 ↓ 10,000,000+

Form Publisher
Generate Google Docs
Form Approvals and Do...
Talarian
Form Publisher is a document generator or document merge solution: generate PDF, Googl...

★ 4.6 ↓ 10,000,000+

GET DATA INTO SHEETS
Awesome Table - Data ...
Talarian
Export data from many sources to Google Sheets™. No technical skill required.

★ 4.5 ↓ 10,000,000+

1. 상단 검색 창에 “colaboratory”를 입력합니다. Co..를 입력하면 자동생성되어 문자가 완성됩니다.
2. 검색 결과에서 colab을 클릭합니다.

1. 파이썬 기반 복습

1.4 파이썬 환경

 Google Workspace Marketplace

앱 검색



Colaboratory

This allows Google Colaboratory to open and create files in Google Drive. It is automatically installed on first use; uninstalling this will not prevent access to Colaboratory.

개발자: [Colaboratory team](#)

정보 업데이트: 2022년 6월 17일

1

설치

호환 기기: 

★★★★★ 3,489 ⓘ

↓ 10,000,000+

개요

사용 권한

리뷰

 Welcome To Colaboratory

File Edit View Insert Runtime Tools Help

Share ⓘ Sign in

1. “설치” 버튼을 클릭하시면 자동으로 설치가 됩니다. 소요시간은 1분 내외입니다.

1. 파이썬 기반 복습

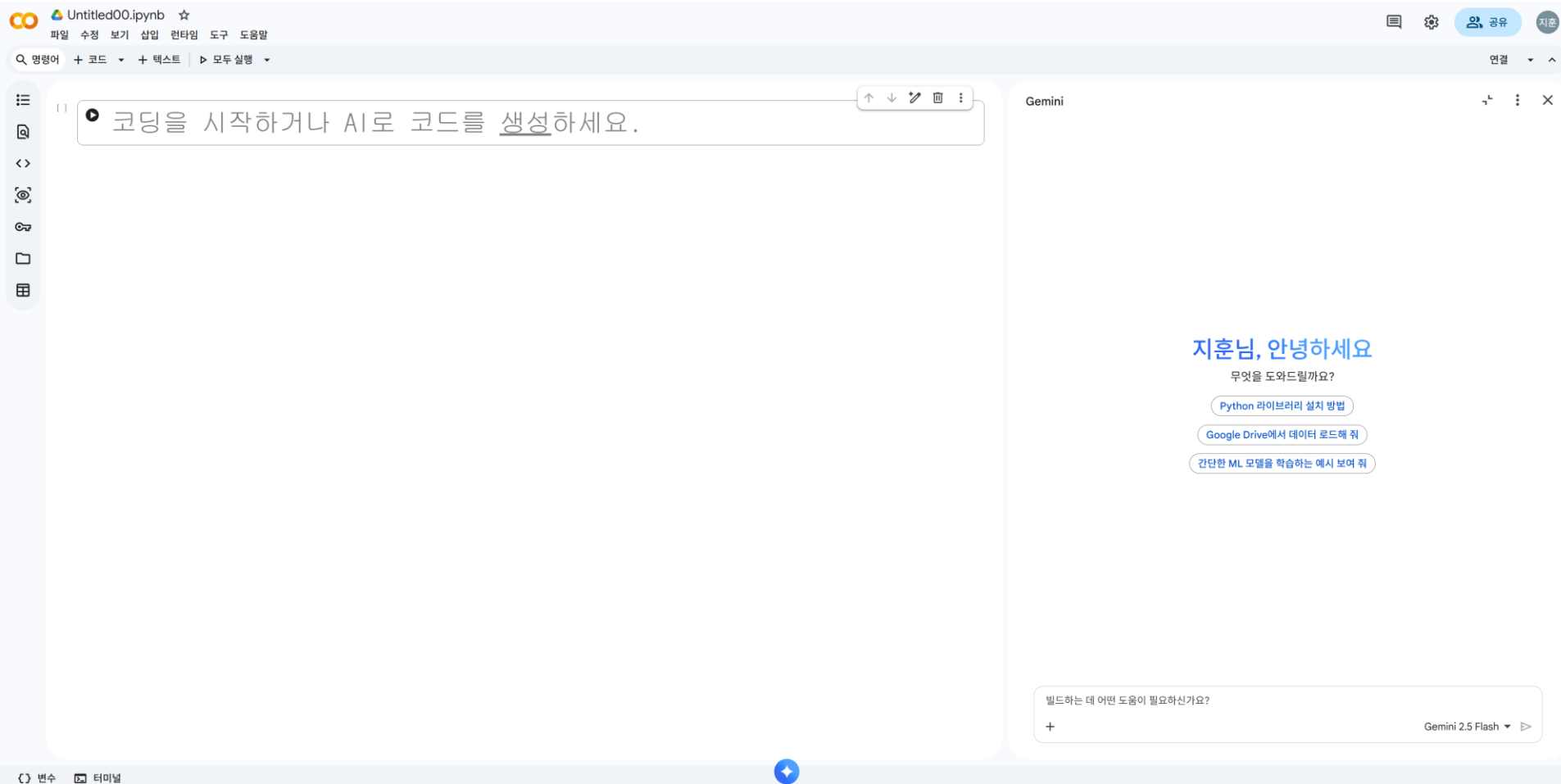
1.4 파이썬 환경

The screenshot shows the Google Drive web interface. On the left, a sidebar menu is open, displaying options like '새 폴더' (New Folder), '파일 업로드' (Upload File), '폴더 업로드' (Upload Folder), and various Google Workspace apps. The '더보기' (More) option is highlighted with a red box and a red circle labeled '1'. A secondary menu is open from '더보기', listing 'Google 드로잉', 'Google 내 지도', 'Google 사이트 도구', 'Google Apps Script', 'Google Colaboratory', and 'Google Jamboard'. 'Google Colaboratory' is highlighted with a red box and a red circle labeled '2'. The main area of the drive shows a message about managing files in one place, with logos for Google Docs, Google Sheets, Google Slides, and Microsoft Office.

- 모든 설치가 완료되었습니다. 불필요한 창은 닫아주세요.
- 1. 다시 드라이브에서 [새로 만들기 -> 더보기]로 커서를 이동합니다.
- 2. Google colaboratory 가 설치된 것을 확인하실 수 있어요. 해당 항목을 클릭합니다.

1. 파이썬 기반 복습

1.4 파이썬 환경

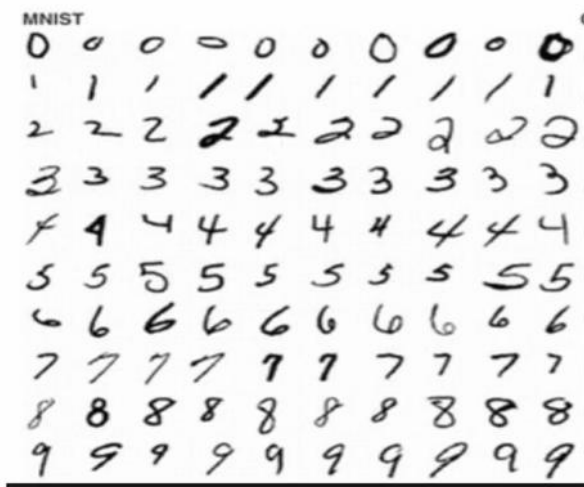


- colab 환경의 메인 화면입니다.
- 구글 코랩(Colab)은 클라우드 기반의 무료 Jupyter 노트북 개발 환경입니다.

1. 파이썬 기반 복습

1.5 파이썬 복습으로 다지기

- 머신러닝은 왜 “데이터 핸들링”이 중요한가?
- 이유1: 일반적인 프로그래밍은 사람이 규칙(if-else)을 직접 정해주지만 머신러닝은 데이터 속에서 컴퓨터가 스스로 규칙을 찾아내기 때문이다.
- 이유2: 예시(MNIST 숫자 인식) 같은 숫자 '3'이라도 사람마다 쓰는 모양이 천차만별이며 이를 수천 개의 if 문으로 정의하는 것은 불가능에 가깝다.
- 이유3: 머신러닝의 방식: 문제(Data)와 답(Label)을 주면, 그 사이의 복잡한 패턴(규칙)을 학습한다.



- 동일한 숫자라 하더라도 여러 변형으로 인해 숫자 인식에 필요한 여러 특징 (feature) 들을 if else 와 같은 조건으로 구분하여 숫자를 인식하기 어렵다.

1. 파이썬 기반 복습

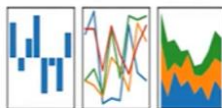
1.5 파이썬 복습으로 다지기

- 데이터 분석의 핵심 도구는 다음과 같으며, 머신러닝 구현에서 데이터의 추출, 가공, 변환은 전체 과정의 80% 이상을 차지한다.
- NumPy (넘파이): 선형대수 기반의 수치 계산을 위한 핵심 라이브러리이다.
- Pandas (판다스): 표(Table) 형태의 데이터를 다루는 데 최적화된 도구이다.
- Scikit-learn (사이킷런): 파이썬의 대표적인 머신러닝 라이브러리이다.



pandas

$$y_{it} = \beta^T x_{it} + \mu_i + \epsilon_{it}$$



1. 파이썬 기반 복습

1.5 파이썬 복습으로 다지기 (넘파이)

- 넘파이의 핵심은 `ndarray(n-dimensional array)`라는 N차원 배열 객체입니다.

✓ 실행 코드

```
import numpy as np # 넘파이 불러오기 (별칭 np 사용)

# 1차원 배열
array1 = np.array([1, 2, 3])

# 2차원 배열 (행과 열)
array2 = np.array([[1, 2, 3], [2, 3, 4]])

print(f"array1 차원: {array1.ndim}, 형태: {array1.shape}") # (3,)
print(f"array2 차원: {array2.ndim}, 형태: {array2.shape}") # (2, 3)
```

array1 차원: 1, 형태: (3,) array2
차원: 2, 형태: (2, 3)

1. 파이썬 기반 복습

1.5 파이썬 복습으로 다지기 (넘파이)

- 넘파이의 핵심은 `ndarray(n-dimensional array)`라는 N차원 배열 객체입니다.

✓ 실행 코드

```
import numpy as np # 넘파이 불러오기 (별칭 np 사용)

# 1차원 배열
array1 = np.array([1, 2, 3])

# 2차원 배열 (행과 열)
array2 = np.array([[1, 2, 3], [2, 3, 4]])

print(f"array1 차원: {array1.ndim}, 형태: {array1.shape}") # (3,)
print(f"array2 차원: {array2.ndim}, 형태: {array2.shape}") # (2, 3)
```

array1 차원: 1, 형태: (3,)
array2 차원: 2, 형태: (2, 3)

1. 파이썬 기반 복습

1.5 파이썬 복습으로 다지기 (넘파이)

- 넘파이는 파이썬에서 선형대수 기반의 수치 계산을 하기 위한 필수 라이브러리이며, 모든 데이터는 숫자의 배열인 ndarray 형태로 표현된다.

✓ 실행 코드

```
import numpy as np

# 1차원 배열: 한 줄로 늘어선 데이터 [shape: (3,)]
array1 = np.array([1, 2, 3])

# 2차원 배열: 행과 열로 구성된 표 형태 [shape: (2, 3)]
array2 = np.array([[1, 2, 3], [2, 3, 4]])

# 특수한 2차원 배열: 1행 3열의 형태 [shape: (1, 3)]
array3 = np.array([[1, 2, 3]])

print(f"array1 차원: {array1.ndim}, 형태: {array1.shape}")
print(f"array2 차원: {array2.ndim}, 형태: {array2.shape}")
print(f"array3 차원: {array3.ndim}, 형태: {array3.shape}")
```

```
array1 차원: 1, 형태: (3, )
array2 차원: 2, 형태: (2, 3)
array3 차원: 2, 형태: (1, 3)
```

1. 파이썬 기반 복습

1.5 파이썬 복습으로 다지기 (넘파이)

- shape 속성은 배열이 몇 행 몇 열로 구성되었는지 알려주는 '데이터의 지도'와 같다.
- 머신러닝 모델에 데이터를 넣을 때 이 형태를 맞추는 것이 가장 중요하다.



✓ 예시 코드

```
array1 = np.array([1,2,3])
print('array1 type:',type(array1))
print('array1 array 형태:',array1.shape)

array2 = np.array([[1,2,3],
                   [2,3,4]])
print('array2 type:',type(array2))
print('array2 array 형태:',array2.shape)
```

1. 파이썬 기반 복습

1.5 파이썬 복습으로 다지기 (넘파이)

✓ 예시 코드 각 배열의 차원 확인하기(ndarray.ndim)

```
array1 = np.array([1,2,3])
print('array1 type:',type(array1))
print('array1 array 형태:',array1.shape)

array2 = np.array([[1,2,3],
                   [2,3,4]])
print('array2 type:',type(array2))
print('array2 array 형태:',array2.shape)
```

```
print('array1: {:0}차원, array2: {:1}차원, array3: {:2}차원'.format(array1.
ndim,array2.ndim,array3.ndim))
```

```
array1 type: <class 'numpy.ndarray'>
array1 array 형태: (3,) array2 type: <class
'numpy.ndarray'> array2 array 형태: (2, 3)
array1: 1차원, array2: 2차원, array3: 2차원
```

1. 파이썬 기반 복습

1.5 파이썬 복습으로 다지기 (넘파이)

- 넘파이 배열 내의 모든 데이터는 반드시 같은 타입이어야 한다.
- 만약 다른 타입이 섞이면 어떻게 될까?

✓ 실행 코드

```
list1 = [1, 2, 'test']  
array_type = np.array(list1)  
print(array_type, array_type.dtype)  
# 결과: ['1' '2' 'test'] <U21 (모두 문자열로 변환됨)
```

`['1' '2' 'test'] <U21`

- 위 코드처럼 정수와 문자열이 섞이면 넘파이는 데이터를 안전하게 보관하기 위해 모두 '문자열'로 바꿔버린다.
- 이를 방지하거나 메모리를 아끼기 위해 `astype()`으로 타입을 직접 바꿀 수 있다.

1. 파이썬 기반 복습

1.5 파이썬 복습으로 다지기 (넘파이)

- 데이터 분석을 하다 보면 0으로 가득 찬 행렬이나 연속된 숫자가 필요할 때가 많다.
- `arange()`: 파이썬의 `range`와 비슷하게 연속된 숫자를 생성
- `zeros()`: 모든 요소를 0으로 채움 (초기화 시 유용)
- `Ones()`: 모든 요소를 1로 채움

✓ 실행 코드

```
sequence_array = np.arange(10) # 0부터 9까지  
zero_array = np.zeros((3, 2), dtype='int32')
```

```
array([[0, 0],  
       [0, 0],  
       [0, 0]], dtype=int32)
```


1. 파이썬 기반 복습

1.5 파이썬 복습으로 다지기 (Reshape)

- 머신러닝 알고리즘은 입력 받는 데이터의 형태(Shape)가 엄격하게 정해져 있다.
- 예를 들어 1차원 데이터를 2차원으로 바꿔야 할 때 reshape()를 사용한다.

✓ 실행 코드

```
array1 = np.arange(10) # 1차원: (10,)
array2 = array1.reshape(2, 5) # 2차원: (2, 5)
# -1의 효과: "열은 5개로 맞추 테니, 행의 개수는 네가 알아서 계산해!"
array3 = array1.reshape(-1, 5)
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
array([[0, 1, 2, 3, 4],
       [5, 6, 7, 8, 9]])
```

```
array([[0, 1, 2, 3, 4],
       [5, 6, 7, 8, 9]])
```

- 주의사항: 전체 데이터 개수가 맞지 않으면 오류가 발생한다. (예: 10개 데이터를 3x3으로 바꿀 수 없음)

2. 데이터 구조 다루기(ndarray)

2.1 ndarray의 동질성

- 파이썬의 일반적인 리스트(List)는 하나의 바구니 안에 숫자, 문자열, 불리언(True/False) 등 서로 다른 종류의 데이터를 마음껏 담을 수 있다. 이 부분이 매우 유연하지만, 데이터를 처리할 때마다 "이 데이터는 무슨 타입이지?"라고 확인하는 과정이 필요해 속도가 느려진다.
- 반면, 넘파이의 ndarray는 "하나의 배열 안에는 반드시 동일한 데이터 타입만 존재해야 한다"는 엄격한 규칙이 있으며 이를 '동질성'이라고 한다.
- 모두가 같은 타입이기 때문에 컴퓨터는 메모리 상에서 데이터가 어디에 있는지, 크기가 얼마인지 즉각 알 수 있어 대규모 연산을 빛의 속도로 처리할 수 있다.

2. 데이터 구조 다루기(ndarray)

2.2 ndarray의 데이터 타입

✓ 실행 코드

```
list2 = [1, 2, 'test']  
array2 = np.array(list2)  
print(array2, array2.dtype)
```

`['1' '2' 'test'] <U21`

```
list3 = [1, 2, 3.0]  
array3 = np.array(list3)  
print(array3, array3.dtype)
```

`[1. 2. 3.] float64`

```
array_int = np.array([1, 2, 3])  
array_float = array_int.astype('float64')  
print(array_float, array_float.dtype)
```

`[1. 2. 3.] float64`

```
array_int1 = array_float.astype('int32')  
print(array_int1, array_int1.dtype)
```

`[1 2 3] int32`

```
array_float1 = np.array([1.1, 2.1, 3.1])  
array_int2 = array_float1.astype('int32')  
print(array_int2, array_int2.dtype)
```

`[1 2 3] int32`

2. 데이터 구조 다루기(ndarray)

2.3 만약 데이터 타입이 섞여 들어오면 어떻게 될까? (Upcasting)

- 만약 실수로 혹은 의도적으로 서로 다른 타입이 섞인 리스트를 넘파이 배열로 만들려고 하면 어떻게 될까?
- 넘파이는 오류를 내뱉는 대신, 더 큰 데이터 타입으로 모든 요소를 통일(Upcasting)시켜 버린다.
- **정수와 실수가 섞이면?** 정수보다 더 넓은 범위를 표현할 수 있는 실수(float)로 모두 바뀐다.

예시: `np.array([1, 2, 3.5])` → `[1.0, 2.0, 3.5]` (정수 1, 2가 1.0, 2.0으로 변함)

- **숫자와 문자열이 섞이면?** 숫자는 계산이 가능하지만 문자열은 불가능하다. 즉, 넘파이는 데이터 손실을 막기 위해 모든 숫자를 문자열(string)로 바꿔버린다.

예시: `np.array([1, 2, '안녕'])` → `['1', '2', '안녕']` (이제 이 숫자들은 더 이상 계산에 쓸 수 없다.)

2. 데이터 구조 다루기(ndarray)

2.4 왜 데이터 타입을 직접 신경 써야 할까? (정답은 메모리 효율)

- 머신러닝에서는 수백만, 수천만 개의 데이터를 다룬다.
- 이때 데이터 타입을 어떻게 정하느냐에 따라 메모리 사용량이 크게 달라질 수 있다.
- ❖ 정수형(int): 8비트, 16비트, 32비트, 64비트 등이 있다. 그리고 숫자가 크지 않는데 64비트를 쓰면 메모리 낭비가 심해진다.
- ❖ 실수형(float): 보통 32비트나 64비트를 쓴다. 정교한 계산이 필요하면 64비트를 쓰지만, 딥러닝 모델을 가볍게 만들 때는 16비트로 줄이기도 한다.
- 배열을 만든 후 현재 타입을 확인하고 싶을 때는 .dtype 속성을 사용한다.

✓ 실행 코드

```
import numpy as np

array = np.array([1, 2, 3])

print(array.dtype) # 결과: int64 (또는 환경에 따라 int32)
```

2. 데이터 구조 다루기(ndarray)

2.5 데이터 타입 바꾸기: astype() 메서드

- 데이터 분석을 하다 보면 정수로 된 데이터를 실수로 바꾸거나, 메모리를 아끼기 위해 타입을 낮춰야 할 때가 있다. 이때 사용하는 함수가 바로 astype()이다.
- 실수를 정수로 바꿀 때: 소수점 아래 자리는 무조건 버려진다. (반올림이 아님을 주의)
- 문자열 숫자를 계산용 숫자로 바꿀 때: 파일에서 읽어온 숫자가 문자열 형태라면 astype('int32')나 astype('float64')로 변환해야 연산이 가능해진다.

✓ 실행 코드

```
# 실수를 정수로 변환하는 예시

float_array = np.array([1.1, 2.9, 3.5])

int_array = float_array.astype('int32')

# 결과는 [1, 2, 3] 이 됩니다.
```

- 넘파이의 데이터 타입은 "빠른 연산을 위해 모두가 같은 옷을 입어야 한다"는 원칙을 가지고 있다.
- 분석가는 데이터의 특성에 맞춰 적절한 옷(dtype)을 입혀줌으로써 계산 효율성과 메모리 절약이라는 두 마리 토끼를 잡을 수 있다. 또한 소수점을 버려도 되는지, 숫자가 얼마나 큰지를 고민하며 타입을 결정하는 것이 데이터 분석가이다.

2. 데이터 구조 다루기(자료구조: 복습)

2.6 List (리스트): 유연한 데이터의 나열

- 리스트는 순서가 있고 수정이 가능한 가장 대표적인 자료구조이다. 데이터 분석에서는 여러 개의 파일명, 칼럼(열) 이름, 혹은 모델의 하이퍼파라미터 목록을 관리할 때 주로 사용한다.

✓ 실행 코드

```
# 리스트 예시

features = ['age', 'income', 'score', 'gender']

features.append('location') # 데이터 추가

print(features[0]) # 'age' (인덱싱)
```

2. 데이터 구조 다루기(자료구조: 복습)

2.7 Tuple (튜플): 변하지 않는 데이터의 약속

- 튜플은 리스트와 비슷하지만, 한 번 만들면 내용을 수정할 수 없다(Immutable). 데이터 분석 결과로 나온 “행렬의 모양(shape)”이나 “함수의 반환값”처럼 변해서는 안 되는 중요한 정보를 담을 때 사용한다.

✓ 실행 코드

```
# 튜플 예시  
  
shape = (100, 4) # 100행 4열을 의미하는 데이터 형태  
  
# shape[0] = 200 <- 실행 시 오류 발생 (수정 불가)
```


2. 데이터 구조 다루기(자료구조: 복습)

2.7 Dictionary (딕셔너리): 키와 값의 매핑

- 딕셔너리는 “Key”와 “Value”가 한 쌍으로 이루어져 있다. 데이터 분석에서 특정 범주형 데이터를 숫자로 바꿀 때(Mapping)나, 판다스 데이터프레임을 생성할 때 핵심적인 역할을 한다.

✓ 실행 코드

```
# 딕셔너리 예시 (범주형 데이터 인코딩)

gender_map = {'male': 0, 'female': 1}

print(gender_map['male']) # 0
```

2. 데이터 구조 다루기(컴프리헨션: 복습)

2.8 List Comprehension

- 컴프리헨션이란 데이터 분석 코드를 보다 간결하고 읽기 쉽게 만들어주는 기법이다. 반복문(for)을 한 줄로 줄여주며 속도도 일반 반복문보다 빠르다.
- 수만 개의 데이터 파일 이름 중에서 특정 확장자만 골라내거나, 숫자에 일괄적으로 연산을 적용할 때 유용하다.

✓ 실행 코드

```
# 일반 반복문

squares = []

for i in range(10):

    squares.append(i**2)


# 리스트 컴프리헨션 (권장 스타일)

squares = [i**2 for i in range(10)]
```

2. 데이터 구조 다루기(컴프리헨션: 복습)

2.9 Dictionary Comprehension

- 두 개의 리스트를 합쳐서 딕셔너리를 만들거나, 기존 딕셔너리의 값을 조건에 따라 가공할 때 사용한다.

✓ 실행 코드

```
# 칼럼명과 인덱스를 매핑하는 딕셔너리 생성

cols = ['name', 'age', 'city']

col_idx = {name: i for i, name in enumerate(cols)}

# 결과: {'name': 0, 'age': 1, 'city': 2}
```

2. 데이터 구조 다루기(데이터 분석 관점의 Python 코드 스타일)

2.10 벡터화(Vectorization) 지향

- 데이터 분석 코드는 나만 보는 것이 아니라 동료나 미래의 나에게 보여주는 '설계도' 이다. 따라서 효율성과 가독성이 조화를 이루어야 한다.
- 분석용 파이썬 코드에서 가장 피해야 할 것은 불필요한 for 문이다. 파이썬의 for 루프는 데이터가 많아질수록 매우 느려진다. 대신, 넘파이(NumPy)나 판다스(Pandas)를 사용하여 데이터 전체를 한꺼번에 계산하는 " 벡터 연산" 스타일을 지향해야 한다.

✓ 실행 코드

```
# [피해야 할 스타일]
import numpy as np
data = np.array([1, 2, 3, 4, 5]) # 예시 데이터 정의
total = 0
for val in data:
    total += val
print(f"For loop total: {total}")
```

```
# [데이터 분석 스타일]
# import numpy as np # 이미 임포트 됨
total_np = np.sum(data) # 훨씬 빠르고 명확함
print(f"Numpy sum total: {total_np}")
```

For loop total: 15
Numpy sum total: 15

2. 데이터 구조 다루기(데이터 분석 관점의 Python 코드 스타일)

2.11 명시적인 변수 이름 사용

- 데이터 분석 과정은 불러오기 -> 전처리 -> 모델링 -> 평가 순으로 진행된다. 이때 변수명을 신경 쓰지 않으면 나중에 어떤 데이터가 원본인지 헷갈리게 된다.

✓ 실행 코드

```
df: 기본 데이터프레임  
df_cleaned: 결측치 등이 처리된 데이터  
X_train: 학습용 특징 데이터  
y_label: 정답 데이터
```

2. 데이터 구조 다루기(데이터 분석 관점의 Python 코드 스타일)

2.12 체이닝(Chaining) 기법의 활용

- 여러 작업을 한 번에 연결해서 수행하면 코드 흐름을 파악하기 쉽다.

✓ 실행 코드

```
# 가독성 높은 스타일

result = (df.dropna()           # 1. 결측치 제거
          .groupby('city')      # 2. 도시별 그룹화
          .mean())              # 3. 평균 계산
```

2. 데이터 구조 다루기(데이터 분석 관점의 Python 코드 스타일)

2.13 라이브러리 호출의 표준 (Convention)

- 데이터 분석 커뮤니티에서 약속된 별칭(Alias)을 사용해야 하며 이는 다른 사람의 코드를 이해하는 시간을 단축해 준다.

✓ 실행 코드

```
import numpy as np  
  
import pandas as pd  
  
import matplotlib.pyplot as plt  
  
import seaborn as sns
```

3. 데이터 구조 다루기(문제 코드로 알아보기)

3.1 [난이도 하] 1단계: 파이썬 기본 자료구조(List + Dict)로 만들기

- 가장 직관적인 방법이며 딕셔너리의 'Key'를 컬럼명으로, 'List'를 해당 컬럼의 데이터로 사용한다.
- 문제 설명:** 엑셀의 한 열(Column)을 하나의 리스트로 보고, 이들을 딕셔너리로 묶은 형태이다. 소규모 데이터를 빠르게 정의할 때 좋습니다.

✓ 실행 코드

1. 순수 파이썬 자료구조를 이용한 학생 성적표 데이터

```
student_data = {  
    'Name': ['김철수', '이영희', '박민지', '최주원'],  
    'Math': [90, 85, 95, 70],  
    'English': [80, 90, 88, 75],  
    'Class': ['A', 'B', 'A', 'C']  
}
```

데이터를 꺼낼 때

```
print(f"학생 명단: {student_data['Name']}")  
print(f"첫 번째 학생의 수학 점수: {student_data['Math'][0]}")
```

학생 명단: ['김철수', '이영희', '박민지', '최주원']

첫 번째 학생의 수학 점수: 90

3. 데이터 구조 다루기(문제 코드로 알아보기)

3.2 [난이도 중] 2단계: 넘파이(NumPy) ndarray로 수치 테이블 만들기

- 넘파이를 사용하면 모든 데이터를 숫자로 변환하여 행렬(Matrix) 구조로 관리한다.
- 계산 속도가 매우 빠르며, shape를 통해 구조를 한눈에 파악할 수 있다.
- **문제 설명:** 2차원 배열 구조이며 각 열이 무엇을 의미하는지는 분석가가 머릿속에 기억하거나 별도의 리스트로 관리해야 한다.

✓ 실행 코드

```
import numpy as np
```

```
# 2. 넘파이를 이용한 4x3 수치 데이터 테이블 (예: 센서 측정값)
```

```
# 행(Row): 측정 시간, 열(Column): [온도, 습도, 조도]
```

```
sensor_readings = np.array([
```

```
    [23.5, 60, 450],
```

```
    [24.1, 58, 455],
```

```
    [23.8, 62, 440],
```

```
    [25.0, 55, 460]
```

```
])
```

```
print("데이터 구조(행, 열):", sensor_readings.shape)
```

```
print("전체 평균 온도:", np.mean(sensor_readings[:, 0])) # 모든 행의 0번 열(온도) 평균
```

데이터 구조(행, 열): (4, 3)

전체 평균 온도: 24.1

3. 데이터 구조 다루기(문제 코드로 알아보기)

3.2 [난이도 중] 2단계: 넘파이(NumPy) ndarray로 수치 테이블 만들기

- 2단계 넘파이 예제에서 온도(0번 열)가 24도 이상인 데이터만 추출할 경우는?

✓ 실행 코드

```
high_temp_data = sensor_readings[sensor_readings[:, 0] >= 24.0]
print(high_temp_data)
```

✓ 실행 결과

```
high_temp_data = sensor_readings[sensor_readings[:, 0] >= 24.0]
print(high_temp_data)
```

```
... [[ 24.1  58. 455. ]
      [ 25.   55. 460. ]]
```

3. 데이터 구조 다루기(문제 코드로 알아보기)

3.3 [난이도 상] 3단계: 불린 인덱싱과 리스트 컴프리헨션 활용

- 데이터를 그냥 만드는 것에 그치지 않고, 특정 조건에 맞는 데이터만 추출하여 새로운 테이블을 만드는 실전 스타일이다.
- 데이터 분석에서 가장 많이 하는 작업이다. Prices $\geq$ 3000과 같은 조건식을 만들어 원하는 데이터만 뽑아내고, 이를 컴프리헨션으로 가공하는 과정이다.

✓ 실행 코드

3. 넘파이와 조건을 활용한 데이터 필터링 테이블

```
names = np.array(['Apple', 'Banana', 'Cherry', 'Date', 'Elderberry'])
```

```
prices = np.array([2500, 1200, 5000, 3500, 4000])
```

```
inventory = np.array([10, 50, 5, 20, 15])
```

조건: 가격이 3000원 이상인 품목만 골라내기 (불린 인덱싱)

```
expensive_mask = prices >= 3000
```

```
expensive_names = names[expensive_mask]
```

```
expensive_prices = prices[expensive_mask]
```

리스트 컴프리헨션으로 비싼 품목 정보 요약 문자열 만들기

```
summary = [f"품목: {n}, 가격: {p}원" for n, p in zip(expensive_names, expensive_prices)]
```

```
print("--- 3000원 이상 품목 요약 ---")
```

```
for item in summary:
```

```
    print(item)
```

```
--- 3000원 이상 품목 요약 ---
```

```
품목: Cherry, 가격: 5000원
```

```
품목: Date, 가격: 3500원
```

```
품목: Elderberry, 가격: 4000원
```



강남대학교
KANGNAM UNIVERSITY

감사합니다

